

Classes, Objects, and Interfaces

A self-guided walkthrough, example by example

This is a hands-on walkthrough, meant for you to work through yourself, not just read. The comprehensive class notes explain every concept in depth, and the slides introduce the vocabulary and motivation; this document is where you actually build the program, one small piece at a time, in the order shown. Type each block of code into your own files, run it, and think through each prompt before moving on. Each example below has a matching folder of finished `.java` files, in case you want to check your work or pick up partway through.

Every listing here should compile exactly as written, and has been run to confirm its output; unlike some lab assignments, nothing here is a TODO or an intentional error for you to find. If you are working in Eclipse, you may need to adjust the layout a little, for example moving a class into the right source folder or package, to get a project to build. Where an example's code carries over and extends the previous example's, the new material is called out so it is easy to see what changed.

1. Example 1: A First Class

Goal. Build one class from nothing, then create and use a couple of objects from it. This is the only example that starts from a blank file.

Create a file named `Car.java` and start with just the fields and a constructor. Before moving on, make sure you can explain what each line does.

```
public class Car {  
    private String make;  
    private String model;  
    private int year;  
    private int mileage;  
  
    public Car(String make, String model, int year) {  
        this.make = make;  
        this.model = model;  
        this.year = year;  
        this.mileage = 0;  
    }  
}
```

Check your understanding. Why does the constructor set `mileage` directly to 0 instead of taking it as a parameter? Every new car should start at zero miles; there is no valid reason to let a caller pick a starting mileage.

Add getters next, then a setter with a private helper behind it. Slow down here and make sure you understand *why* the helper exists, not just what it does.

```
public String getMake() { return make; }
```

```

public String getModel() { return model; }
public int getYear()     { return year; }
public int getMileage()  { return mileage; }

public void setMileage(int newMileage) {
    if (isValidMileage(newMileage)) {
        mileage = newMileage;
    } else {
        System.out.println("Ignored an invalid mileage: " + newMileage);
    }
}

private boolean isValidMileage(int candidate) {
    return candidate >= mileage;
}

public void drive(int miles) {
    setMileage(mileage + miles);
}
}

```

Predict before you run it. What happens if `drive` is called with a negative number? Write down your guess, then run it and check. `isValidMileage` rejects it, since the result would be less than the current mileage.

Now create a second file, for example `Example1Demo.java`, and instantiate a couple of cars.

```

Car civic = new Car("Honda", "Civic", 2022);
Car mustang = new Car("Ford", "Mustang", 2024);

civic.drive(120);
mustang.setMileage(15);

System.out.println(civic.getMake() + " " + civic.getModel()
    + ": " + civic.getMileage() + " miles");
System.out.println(mustang.getMake() + " " + mustang.getModel()
    + ": " + mustang.getMileage() + " miles");

civic.setMileage(10);    // watch this get rejected

```

Close this example with the identity question, since it sets up hash codes later (Example 4).

```

Car civic2 = new Car("Honda", "Civic", 2022);
System.out.println(civic == civic2);    // false
System.out.println(civic == civic);     // true

```

Check your understanding. `civic` and `civic2` have identical make, model, and year. Why does Java still say they are not equal? Sit with that question for a moment; it is exactly the motivation for Example 4, where you will fix it.

Try it yourself. Add a third field to `Car`, for example a VIN string. Decide for yourself whether

it belongs in the constructor, gets its own setter, or both, and be ready to explain your choice.

2. Example 2: A Second Class, and How Objects Relate

Goal. Keep `Car.java` from Example 1 unchanged, and build two more classes around it, so you see a program made of several cooperating objects for the first time.

Build `Mechanic` first, since it is the simpler relationship.

```
public class Mechanic {
    public void inspect(Car car) {
        System.out.println("Inspecting a " + car.getYear() + " "
            + car.getMake() + " " + car.getModel());
    }
}
```

Check your understanding. Does `Mechanic` store a `Car` anywhere? It does not; it borrows one for the duration of a single method call. This is dependence, uses-a.

Then build `Garage`, which is the stronger relationship.

```
import java.util.ArrayList;

public class Garage {
    private String location;
    private ArrayList<Car> fleet;

    public Garage(String location) {
        this.location = location;
        this.fleet = new ArrayList<>();
    }

    public void addCar(Car car) {
        fleet.add(car);
    }

    public Car findByModel(String model) {
        for (Car car : fleet) {
            if (car.getModel().equalsIgnoreCase(model)) {
                return car;
            }
        }
        return null;
    }

    public void printFleet() {
        for (Car car : fleet) {
            System.out.println(car.getYear() + " " + car.getMake()
                + " " + car.getModel());
        }
    }
}
```

Check your understanding. What is different about how `Garage` holds on to its `Car` objects, compared to `Mechanic`? It keeps them in a field across many method calls, rather than borrowing one for a single call. This is aggregation, has-a. A related question worth asking yourself: could a `Car` exist without a `Garage`? It can, which is exactly the independent-lifecycle property that separates aggregation from composition. See the class notes, Section 6, for the fuller version of this point.

Now put it all together in a driver class.

```
Garage shop = new Garage("Main Street");
shop.addCar(new Car("Honda", "Civic", 2022));
shop.addCar(new Car("Ford", "Mustang", 2024));
shop.addCar(new Car("Toyota", "Corolla", 2020));

shop.printFleet();

Car found = shop.findByModel("Mustang");
Mechanic tom = new Mechanic();
if (found != null) {
    tom.inspect(found);
}
```

Try it yourself. Add a `removeCar(String model)` method to `Garage` on your own, using `findByModel` as a model for the loop.

3. Example 3: Passing Objects, and a List of Objects

Goal. `Car`, `Mechanic`, and `Garage` carry over unchanged from Example 2. This example is about two habits: what actually happens when an object is passed to a method, and using `ArrayList` with a class of your own instead of a built-in type.

Start with the pass-by-value puzzle, since it rewards being run rather than just read.

```
public static void serviceCar(Car car) {
    car.setMileage(car.getMileage() + 5000);
}

public static void replaceCar(Car car) {
    car = new Car("Toyota", "Corolla", 2020);
}

Car civic = new Car("Honda", "Civic", 2022);

serviceCar(civic);
System.out.println(civic.getMileage());    // 5000

replaceCar(civic);
System.out.println(civic.getModel());      // still "Civic"
```

Predict before you run it. Will `civic.getModel()` print "Civic" or "Corolla" after `replaceCar` runs? Write down your answer, then run it. What actually gets passed is a copy

of the reference, not the object; reassigning the parameter inside the method only redirects that local copy.

The **Garage** fleet from Example 2 is already an **ArrayList** of objects, so glance back at it, then look at the same pattern applied to something less expected. *[Pokemon]*

```
import java.util.ArrayList;

ArrayList<Pokemon> party = new ArrayList<>();
party.add(new Pokemon("Pikachu", "Electric", 35));
party.add(new Pokemon("Charizard", "Fire", 78));
party.add(new Pokemon("Snorlax", "Normal", 160));

for (Pokemon p : party) {
    System.out.println(p.getName() + " (" + p.getType()
        + "), HP: " + p.getHp());
}

int totalHp = 0;
for (Pokemon p : party) {
    totalHp += p.getHp();
}
System.out.println("Total party HP: " + totalHp);
```

Pokemon is a tiny class with just a name, a type, and an HP field, built the same way **Car** was in Example 1. The point to take away: **ArrayList** does not care what the object is.

Try it yourself. Would sorting the party by HP be any different from sorting the fleet by mileage? Think it through, then try writing one of the two. The pattern is identical either way.

4. Example 4: Overriding toString, equals, and hashCode

Goal. Revisit the Example 1 identity puzzle and resolve it, by overriding the three methods every class inherits from **Object**.

Add the three overrides to your **Car.java** one at a time, running your demo after each one so the change is visible immediately.

```
@Override
public String toString() {
    return year + " " + make + " " + model + " (" + mileage + " miles)";
}
```

Run it: `System.out.println(civic)` now prints something a human can read, instead of a class name and a hash code.

```
@Override
public boolean equals(Object other) {
    if (this == other) {
        return true;
    }
    if (!(other instanceof Car)) {
```

```

        return false;
    }
    Car otherCar = (Car) other;
    return year == otherCar.year
        && make.equals(otherCar.make)
        && model.equals(otherCar.model);
}

```

Run it: the Example 1 identity example again. `civic == civic2` is still **false**, but `civic.equals(civic2)` is now **true**. Make sure you can state the distinction in your own words: `==` compares identity, `equals` (once overridden) compares state.

```

import java.util.Objects;

@Override
public int hashCode() {
    return Objects.hash(make, model, year);
}

```

Check your understanding. Why does `hashCode` use the exact same three fields as `equals`? Two objects that `equals` says are equal must report the same hash code, or code that relies on hashing, such as a `HashSet` later in the course, breaks in ways that are hard to trace.

Try it yourself. What would go wrong if `equals` compared `mileage` too? Try adding it and see what breaks. Two otherwise-identical cars would stop being equal the moment either one was driven, which is almost never what you want from “the same car.”

5. Example 5: Interfaces

Goal. `Car` carries over from Example 4 with its three overrides intact. This example adds an interface and makes `Car` implement it.

Build the interface first, on its own.

```

public interface Serviceable {
    int MAX_MILES_BETWEEN_SERVICES = 5000;

    void service();

    default void honk() {
        System.out.println("Beep beep!");
    }
}

```

Check your understanding. Which of these three members needs a body written by every class that implements `Serviceable`, and which one already has one? `service()` is abstract and needs a body; `honk()` is default and already has one; the constant is shared automatically.

Change the `Car` class header to `implements Serviceable`, and add the one method the interface requires.

```

public class Car implements Serviceable {
    // ... existing fields, constructor, and methods ...
    private int milesSinceService;    // new field for this method

    @Override
    public void service() {
        if (milesSinceService >= MAX_MILES_BETWEEN_SERVICES) {
            System.out.println(model + " was overdue and has now been serviced.");
        } else {
            System.out.println(model + " has been serviced.");
        }
        milesSinceService = 0;
    }
}

```

A mistake worth making on purpose. Try writing `setMileage(0)` inside `service()` instead, as a shortcut for “resetting” something. Run it and watch `isValidMileage` reject it, since the odometer is never allowed to go backward. This is a real bug I ran into while writing this exact example, and working through why it fails yourself is more memorable than being told the right answer up front. The fix is the separate `milesSinceService` field shown above, which tracks something that is genuinely allowed to reset.

Finish with a demo of your own.

```

Car civic = new Car("Honda", "Civic", 2022);
civic.drive(6000);

civic.service();
civic.honk();

System.out.println("Service interval: "
    + Serviceable.MAX_MILES_BETWEEN_SERVICES + " miles");

```

Wrapping up. You now have working vocabulary for three ways objects relate: uses-a (Example 2’s *Mechanic*), has-a (Example 2’s *Garage*), and implements (this example’s *Serviceable*). The natural next question is is-a, inheritance, which is previewed in the slides and gets its own unit later in the semester.